

STAGE 1: SEARCH ENGINE DATA LAYER & BENCHMARK

Author: Pablo Martínez Suárez | Language: C++17 (Clang/GCC -O2) | Course: Big Data — ULPGC

1. Executive Summary & Architecture Overview

This technical report documents the complete implementation, benchmarking, and empirical analysis of the **Stage 1 Data Layer** for an autonomous search engine written in **C++17**. The system strictly adheres to the *Cross-Language Benchmark Contract* (CONTRACT.md), enabling direct, 1:1 comparable evaluations against Java and Python counterparts. The architecture is cleanly decoupled into four core subsystems: **(1) Datalake Storage Engine** supporting three distinct directory layout topologies with single-pass incremental detection and idempotent recovery; **(2) Metadata Datamart** utilizing SQLite with transaction-batched WAL ingestion (> 550,000 rows/s); **(3) Inverted Index Datamarts** comparing Monolithic JSON, Hierarchical Sharded Folders, and a Relational SQLite Index with true in-memory RAM profiling; and **(4) Control Layer** orchestrating pipeline ingestion via libcurl and automated multi-datamart state synchronization.

2. Datalake Multi-Scale Evaluation (100 / 1,000 / 10,000 Books)

We evaluated three physical partitioning strategies across scales of 100, 1,000, and 10,000 books: **Time-based** (YYYYMMDD/HH/), **Book-based** (<id>/), and **Batch-based** (batch_N/, 500 books/batch). Metrics include initial write throughput, random lookup latency, single-pass incremental detection time, incremental write time, idempotent recovery duration, and directory metadata overhead.

Scale	Structure	Write (s)	Books/s	Lookup (ms)	Incr. Det (s)	Incr. Wrt (s)	Recovery (s)	Dirs
100	time_based	0.099	1,009.2	0.011	0.0002	0.013	0.035	2
100	book_based	0.274	365.1	0.004	0.0001	0.015	0.019	100
100	batch_based	0.105	955.7	0.004	0.0001	0.009	0.009	1
1,000	time_based	1.121	891.9	0.016	0.0013	0.127	0.155	2
1,000	book_based	1.201	832.8	0.013	0.0036	0.123	0.139	1,000
1,000	batch_based	1.457	686.3	0.012	0.0031	0.108	0.142	2
10,000	time_based	12.048	830.0	0.019	0.0114	1.180	5.305	2
10,000	book_based	12.717	786.3	0.013	0.0413	1.191	4.745	10,000
10,000	batch_based	11.554	865.5	0.017	0.0361	1.034	3.916	20

Datalake Analysis & Findings:

- *Write Throughput & Inode Overhead*: time_based and batch_based achieve the highest throughput (~830–1,000 books/s) by minimizing directory creations. book_based creates 10,000 directories at scale, introducing significant filesystem inode table pressure.
- *Incremental Processing & Recovery*: Single-pass scan (detectNewBooks) takes only **11.4 ms** for 10,000 books in time_based, versus 41.3 ms in book_based due to directory tree recursion overhead. Recovery idempotence verified: 100% of missing files were restored with zero duplicate corruption.

3. Metadata SQLite Datamart Benchmark (Team Extension)

Project Gutenberg headers were parsed via C++ std::regex into structured fields (book_id, title, author, language, ingested_at, header_path, body_path) and inserted into SQLite using explicit transaction chunking (BEGIN TRANSACTION / COMMIT) with prepared statements. Lookups were tested for exact PK (find_by_id) and partial match (find_by_author) using *Charles Dickens* (repeated) and *Lewis Carroll* (unique).

Scale (Books)	Insert (s)	Throughput (rows/s)	find_by_id (ms)	Dickens (ms)	Carroll (ms)	DB Size (KB)
100	0.0173	5,769.5	0.200	0.203	0.153	28.0
1,000	0.0034	298,188.5	0.148	0.288	0.242	168.0

10,000	0.0180	555,041.6	0.146	1.448	1.093	1,564.0
--------	--------	-----------	-------	-------	-------	---------

Metadata Findings: Transactional batching delivers outstanding write scalability, climbing to **555,041 rows/sec** at 10,000 books. Primary key index lookups remain constant at **~0.14 ms** ($O(1)$ /\$O(\log N)\$ B-Tree), while unindexed author substring scans scale linearly with dataset cardinality.

4. Inverted Index Datamart Multi-Scale Evaluation & Memory Profiling

Following the shared contract, we indexed the corpus using the exact tokenization rule (lowercase ASCII [A-Za-z]+) and evaluated three architectures: **1. Monolithic JSON** (single file containing all vocabulary postings loaded into RAM), **2. Hierarchical Sharded** (directory-partitioned term files A-Z/_ with zero in-memory residency), and **3. Relational SQLite Index** (CREATE TABLE inverted_index(term TEXT, book_id INTEGER, PRIMARY KEY (term, book_id)) with page cache buffer). RAM reflects active in-memory resident structure consumption during query execution.

Scale	Structure	Build (s)	Lookup avg (ms)	Update (ms)	Active RAM	Disk (MB)
100	json_monolithic	0.060	0.001	136.5	6.2 MB	8.66
100	hierarchical	3.333	0.121	0.439	64 KB	4.67
100	sqlite_index	0.738	0.185	0.739	2.0 MB	40.13
1,000	json_monolithic	0.426	0.001	1,011.7	30.8 MB	80.69
1,000	hierarchical	3.635	0.188	1.650	64 KB	46.69
1,000	sqlite_index	7.645	0.429	0.850	2.0 MB	406.78
10,000	json_monolithic	5.684	0.001	13,492.8	276.7 MB	801.05
10,000	hierarchical	7.842	1.319	4.175	64 KB	466.89
10,000	sqlite_index	82.813	10.041	1.816	2.0 MB	4,225.46

- Inverted Index Architectural Trade-offs & Memory Mechanics:**
- **Monolithic JSON:** Fastest in-memory query lookup (**~0.001 ms**) when preloaded, but memory footprint in RAM grows linearly with dataset vocabulary (**6.2 MB → 30.8 MB → 276.7 MB**). It suffers fatal incremental update degradation (**13.5 seconds** per book at 10,000 scale) due to complete file rewriting.
 - **Hierarchical Index:** Zero in-memory residency (constant **64 KB** streaming buffer footprint) and localized atomic updates (**4.1 ms**), completely avoiding global lock contention.
 - **SQLite Relational Index:** Guarantees full ACID transactional integrity with a fixed page cache buffer (**2.0 MB**). Incremental updates remain exceptionally fast and constant (**1.8 ms**) even at 10,000 books.

5. Automated Testing Suite & Contract Compliance

The C++ module includes a comprehensive automated unit test suite integrated into CMake (ctest) and standalone binary execution. **16 distinct test suites with 119 individual assertions** verify: (1) Tokenizer regex & ASCII compliance; (2) Datalake path generation, multi-layout saving, single-pass incremental change detection, and recovery idempotence; (3) Metadata extraction and SQLite CRUD transactions; (4) Inverted Index construction, search correctness, and **cross-structure consistency** (identical results across JSON, Hierarchical, and SQLite); (5) QueryEngine Boolean AND/OR operations; and (6) Control Layer state machine transitions. **Current Test Results: 16 passed, 0 failed (100% pass rate).**

6. Cross-Language Conclusions & Summary

The C++ implementation demonstrates near-zero runtime overhead, exceptional raw I/O throughput, and sub-millisecond query execution. By aligning tokenization rules, query workloads, and synthetic scaling factors via CONTRACT.md, the C++ results establish an exact empirical baseline for comparing Java and Python in the final cross-language benchmark evaluation.